Hire-a-Helper

Description

Hire-a-Helper is an intuitive, robust, and scalable web-based platform designed to bridge the gap between businesses or individuals who need help with everyday tasks and skilled professionals (or everyday people) looking to earn money by providing their services. The platform serves as a localized, on-demand task marketplace.

It empowers users (Task Owners) to efficiently post tasks, evaluate incoming applicants, and manage ongoing work seamlessly. Simultaneously, it provides Helpers an avenue to showcase their talents, browse a dynamic feed of tasks, filter them according to their expertise and location, and find opportunities tailored to their capabilities. Using advanced matching feed, robust request management, and real-time availability tracking, the platform drastically reduces the time and effort required for task allocation and freelance hiring. Key features include dynamic feed, user-friendly profiles with integrated task histories, and a seamless dashboard for managing applications, ongoing projects, and system notifications.

Purpose

The core purpose of Hire-a-Helper is to simplify, organize, and enhance the task-matching process for both everyday users and freelancers by:

- **Optimizing Talent Discovery:** Enable task owners to identify the right candidates quickly based on skills, experience, location, and real-time availability.
- **Empowering Professionals:** Provide individuals and gig-workers with tools to present their expertise, track their requests, and gain access to tasks aligned with their capabilities.
- **Streamlining Processes:** Replace traditional, time-intensive, and unorganized hiring workflows with a smart, technology-driven approach.
- **Encouraging Collaboration:** Foster connections between local communities and professionals, facilitating a mutually beneficial economic ecosystem.

By addressing the inefficiencies in traditional recruitment and resource allocation processes, Hire-a-Helper aims to become a go-to platform for short-term hiring and project-based collaboration.

Getting Started

Prerequisites

Before setting up and running the application locally, ensure the following software and tools are installed on your system:

- **Node.js and npm**
  - Download and install from the Node.js official website.
  - `npm` comes bundled with Node.js.
- **MongoDB**
  - Install and set up MongoDB locally or use a cloud-hosted solution like MongoDB Atlas.
- **SMTP Service**
  - Configure an SMTP service (e.g., Gmail SMTP) for sending OTPs and notifications.
- **IDE**
  - Use a development environment like Visual Studio Code for code editing and debugging.

Installation

Follow these exact steps to clone and set up the project:

1. Clone the Repository

```
git clone https://github.com/springboardmentor210/HireHelper_Team1.git
cd HireHelper_Team1
```

2. Install Dependencies

```
cd server
npm install
cd../client
npm install
```

Running the Application

- **Start the Backend**

  Navigate to the backend directory:

  ```
  cd server
  ```

  Run the backend server (ensure the backend runs on port 5000):

```
npm run dev
```

- **Start the Frontend**

Navigate to the frontend directory:

```
cd client
```

Run the development server:

```
npm run dev
```

Accessing the Application

- **Frontend:** Visit http://localhost:5173 (or the port specified by Vite in the terminal).
- **Backend API:** Accessible at http://localhost:5000 for testing endpoints.

Environment Setup for Hire-a-Helper

To run the project seamlessly, you need to set up environment variables for the backend. Create a `.env` file in the root directory of the backend project (`server/.env`) and add the required configurations as shown below.

Required Environment Variables

Variable Name	Description	Example Value
`PORT`	Port for the backend server	`5000`
`MONGO_URI`	Connection string for MongoDB database	`mongodb://localhost:27017/HireHelper`
`JWT_SECRET`	Secret key for signing JWT tokens	`supersecretkey123`
`EMAIL_USER`	Email address used for sending emails	your-email@example.com
`EMAIL_PASS`	Password or app-specific password for the email	`your-email-password`

Example .env Configuration

Here's an example `.env` file for your project:

Server Configuration

PORT=5000

Database Configuration (MongoDB Atlas)

MONGO_URI=mongodb+srv://:@cluster.mongodb.net/HireHelper

JWT Authentication Secret

JWT_SECRET=your_jwt_secret_key_here

Nodemailer Configuration (For OTP and Password Reset)

EMAIL_USER=your_email@gmail.com EMAIL_PASS=your_16_digit_app_password

Steps to Set Up the Environment

1. Create a `.env` file in the root of your backend directory.
2. Copy the example configuration above and paste it into the `.env` file.
3. Replace placeholder values with your actual configuration.

Additional Notes

- For cloud-hosted MongoDB (e.g., MongoDB Atlas), replace the URI with your cluster's connection string.
- Keep the `.env` file secure and do not commit it to version control. Add `.env` to your `.gitignore` file.

Folder Structure

A well-organized folder structure helps maintain clarity and efficiency in development. Below is the folder structure for the Hire-a-Helper project, leveraging the MERN stack.

```
HireHelper_Team1/

│
├── client/                    # React Frontend (Vite)
│   ├── public/                # Static assets
│   ├── src/
```

```
│   │   ├── assets/            # Images and styles
│   │   ├── components/        # Reusable UI components
│   │   ├── context/           # React Context (State Management)
│   │   ├── features/          # Feature-based modules (Auth, Dashboard, Feed,
Tasks)
│   │   ├── services/          # Axios API calls to the server
│   │   ├── App.jsx            # Main Routing component
│   │   └── main.jsx           # React Entry point
│   └── package.json           # Frontend dependencies
│
└── server/                    # Node.js + Express Backend
    ├── src/
    │   ├── config/            # Database connection (db.js)
    │   ├── controllers/       # Request handlers (auth, dashboard, task,
request)
    │   ├── middleware/        # Custom middlewares (auth, upload)
    │   ├── models/            # Mongoose schemas (User, Task, Request)
    │   ├── routes/            # Express API routes
    │   ├── services/          # Business logic (email, OTP)
    │   ├── utils/             # Helper functions (ApiError, ApiResponse)
    │   ├── app.js             # Express app setup
    │   └── server.js          # Server entry point
    ├── .env                   # Environment variables

    └── package.json           # Backend dependencies
```

Key Features

- **Dynamic Matching Algorithm:** Matches helpers to tasks based on categories, skills, and availability.
- **Helper Profiles:** Comprehensive profiles for users to showcase their details.
- **Task Posting and Management:** Users can post tasks with specific requirements, manage active listings, and track applications.
- **Application Tracking:** Real-time application tracking for both helpers and task owners.
- **Notifications System:** In-app notifications for task updates, request status, and reminders.
- **Secure Authentication:** User authentication using JWT for secure login and session management.
- **Responsive Design:** Frontend designed to be mobile and desktop-friendly for a seamless user experience.

Project Components

Here's a detailed list of key components in the Hire-a-Helper project, along with descriptions of their functionality:

Landing Page Component

Description: Serves as the initial entry point for all unauthenticated users visiting the Hire-a-Helper platform.

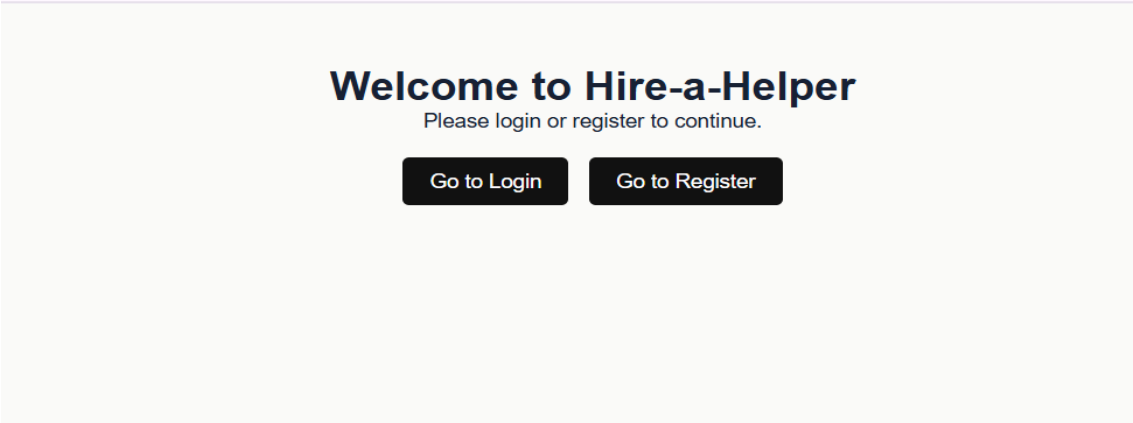
Key Features:

Clear Call-to-Action (CTA): Prominently displays two highly visible buttons encouraging new users to "Go to Register" and returning users to "Go to Login".

Minimalist Interface: Keeps the user focused entirely on the onboarding process without unnecessary distractions.

Welcome Message: Greets the users and clearly states the required actions to proceed into the application.

Routing Gateway: Acts as the primary gateway protecting the internal application routes, ensuring users are authenticated before accessing the feed or dashboard.



Landing Page Interface

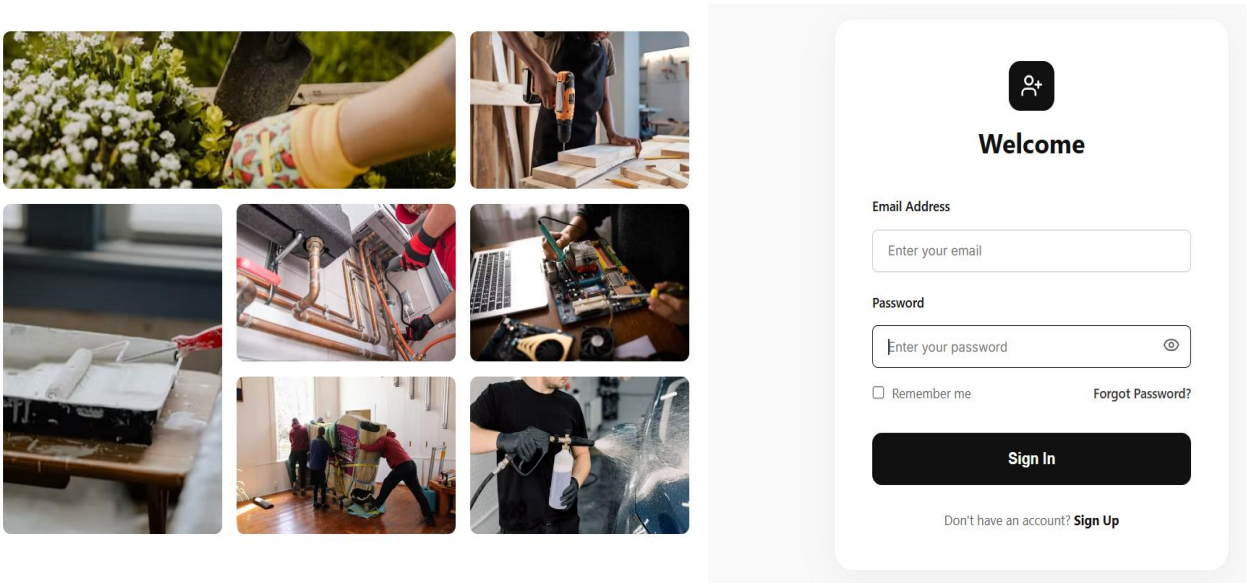
Sign In Component

Description: Manages the user login process, handling inputs for email and password.

Key Features:

- **Password Visibility Toggle:** Allows users to show/hide their password for easy input.
- **'Remember Me' Functionality:** Saves user sign in credentials for subsequent visits without needing to re-enter credentials.

- **Form Validation:** Ensures that the user provides valid credentials before submission.
- **Error Handling:** Displays error messages in case of incorrect credentials.



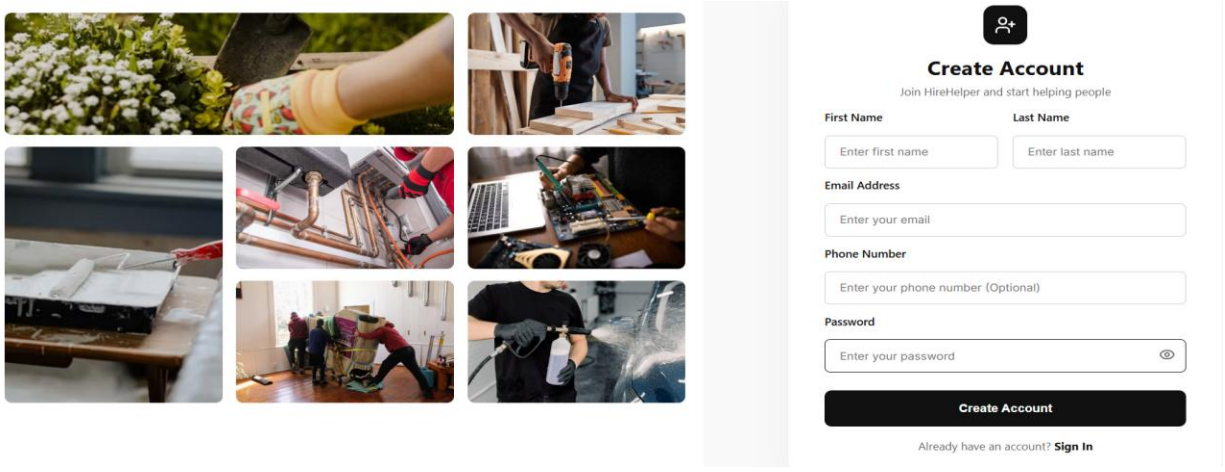
Sign In Interface

Register Component

Provides signup functionality for both helpers and task owners.

Key Features:

- **Form Fields:** Includes fields for first name, last name, email, phone number, and password.
- **Validation:** Ensures that all necessary fields are filled out correctly and securely.
- **Submit & Success Handling:** On successful registration, redirects users to the OTP verification component.



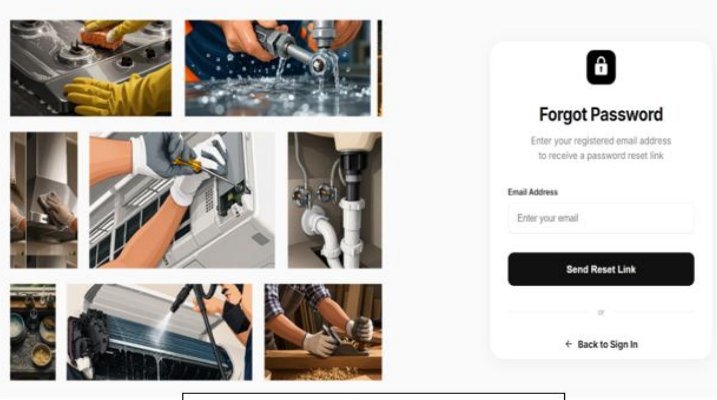
Registration Interface

Forgot Password, OTP & Reset Component

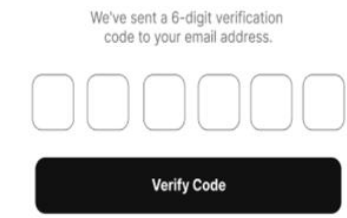
Description: Implements the complete flow for users who have forgotten their password, allowing them to safely recover their account using email verification.

Key Features:

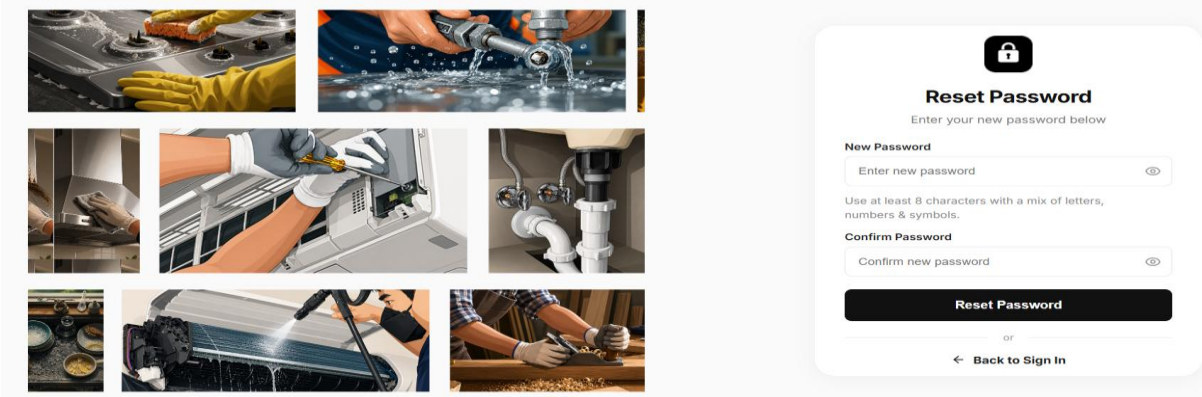
- **Forgot Password Request:** Allows users to easily request a password recovery by entering their registered email address.
- **OTP Generation:** Generates a secure one-time password (OTP) and sends it directly to the user's email via Gmail SMTP.
- **OTP Validation:** Users input the OTP received in their email to securely verify their identity.
- **Password Reset:** Allows users to type and confirm a new password after successful OTP verification.



Forgot Password Request Screen



OTP Verification Interface



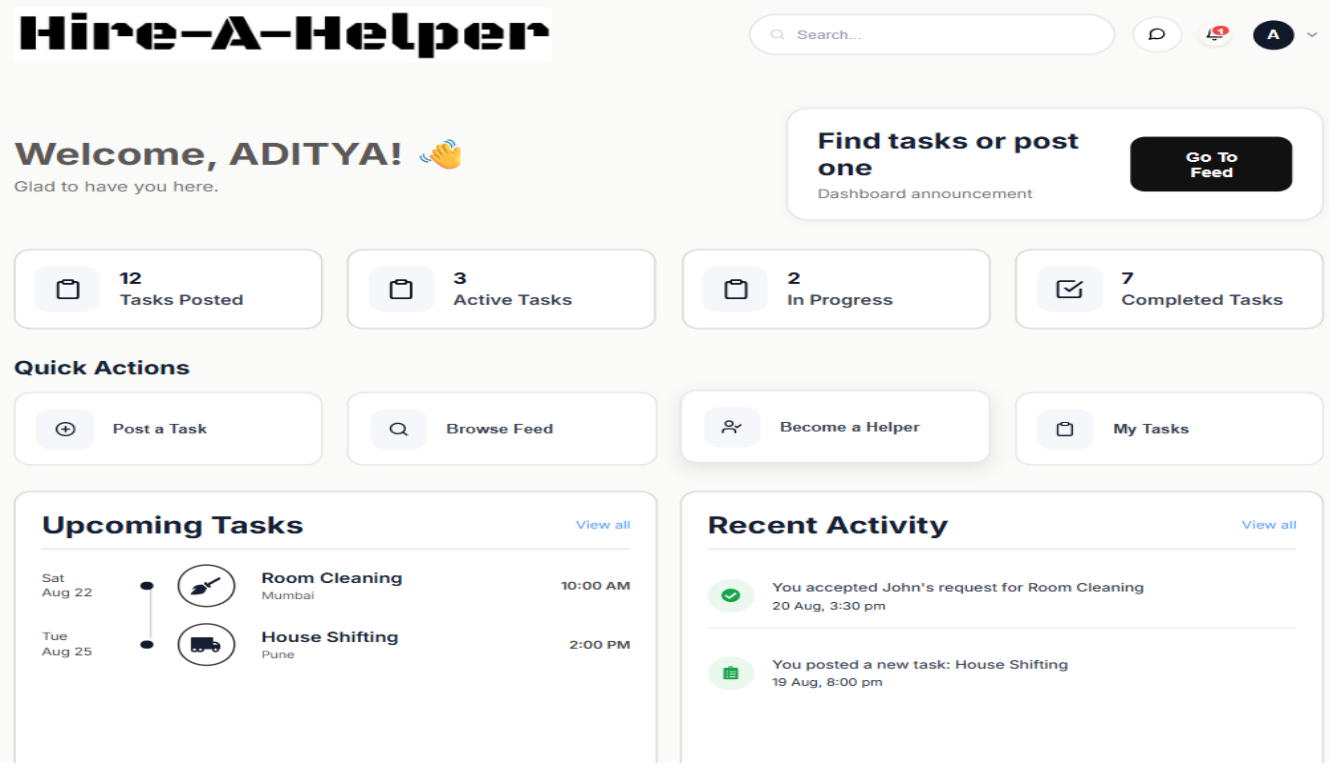
Reset and Confirm Password Interface

Dynamic Dashboard Views

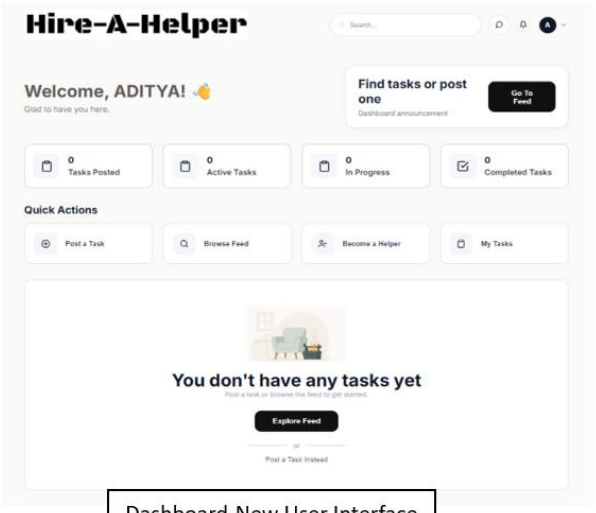
Description: The platform features a dynamic, state-aware dashboard that adapts its interface based on the user's authentication status and past activity on the platform.

Key Features:

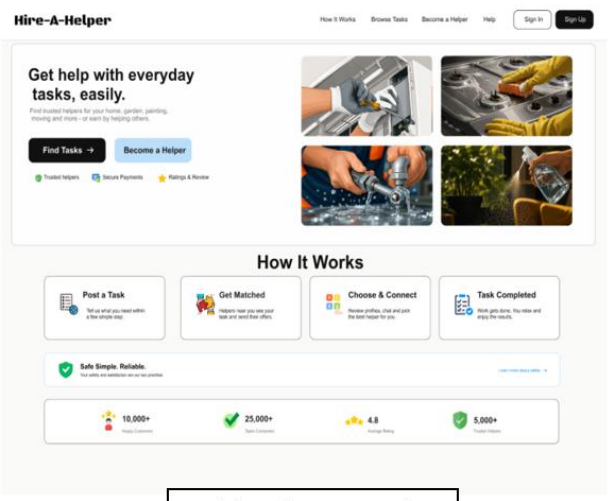
- **Guest View (Landing):** Unauthenticated users see a welcoming "How it Works" interface, encouraging them to sign up, explore features, or find tasks.
- **New User View (Empty State):** Freshly registered users are greeted with a personalized welcome, quick action buttons, and an intuitive "empty state" inviting them to post their first task.
- **Returning User View:** Active users get a comprehensive dashboard populated with their live data, including 'Upcoming Tasks', 'Recent Activity' timelines, and earnings summaries.



Dashboard-Returning User Interface



Dashboard-New User Interface



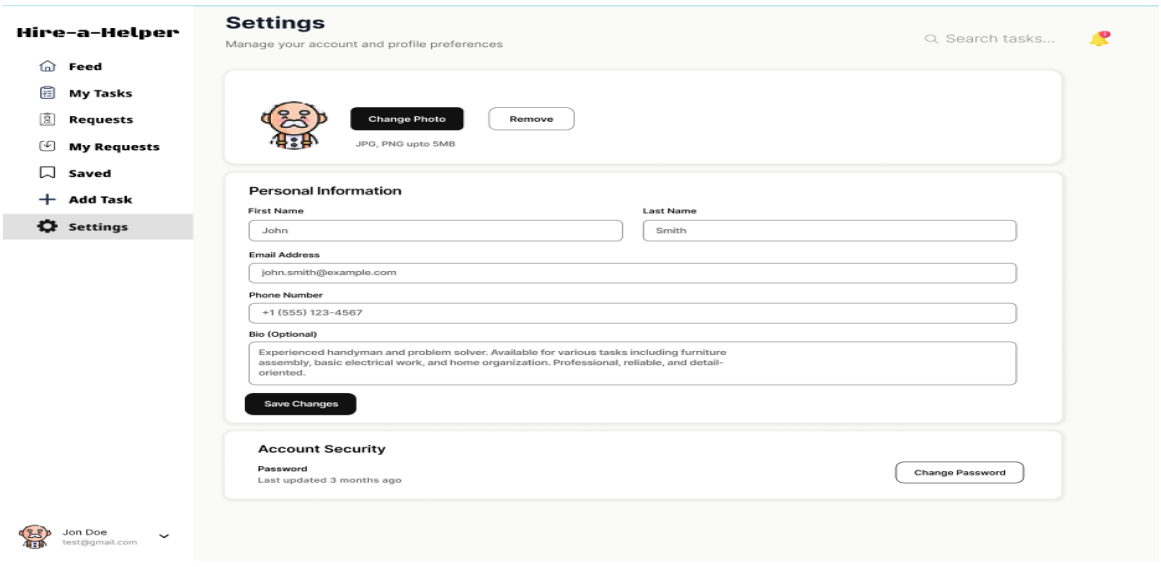
Dashboard-Guest Interface

Helper Profile Component (Settings)

Description: Allows helpers to create, manage, and update their personal and professional details.

Key Features:

- **Profile Image Upload:** Users can upload and preview a profile image.
- **Editable Fields:** Supports editing personal details like name, email, phone, and address.
- **Validation:** Ensures all fields are correctly filled.
- **Success Notification:** Displays confirmation upon successful updates.



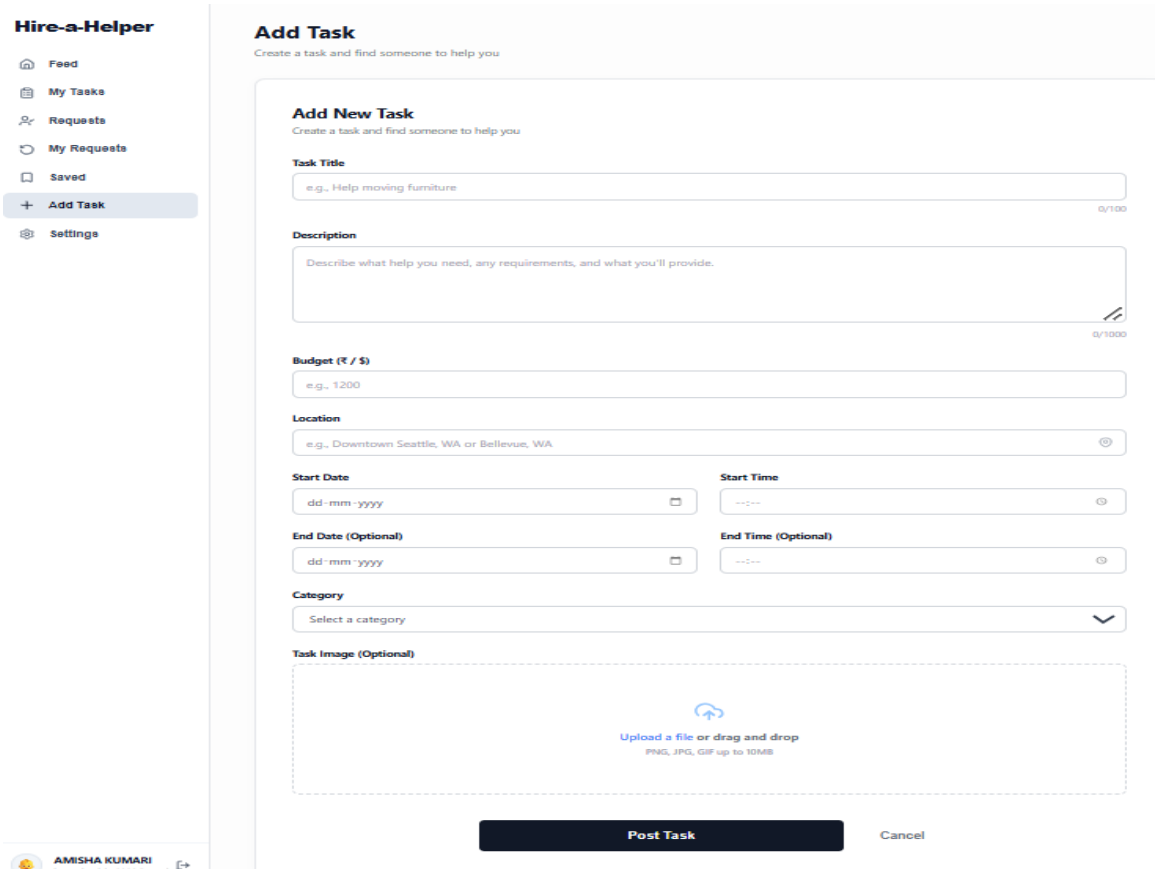
Settings Interface

Task Posting Component (Add Task)

Manages task listings, enabling users to create, edit, and manage task opportunities.

Key Features:

- **Task Listing Creation:** Users can create task posts by entering details like title, description, budget, and requirements.
- **Task Categories:** Users can categorize task posts (e.g., Moving, Plumbing, Cleaning) for easier filtering.



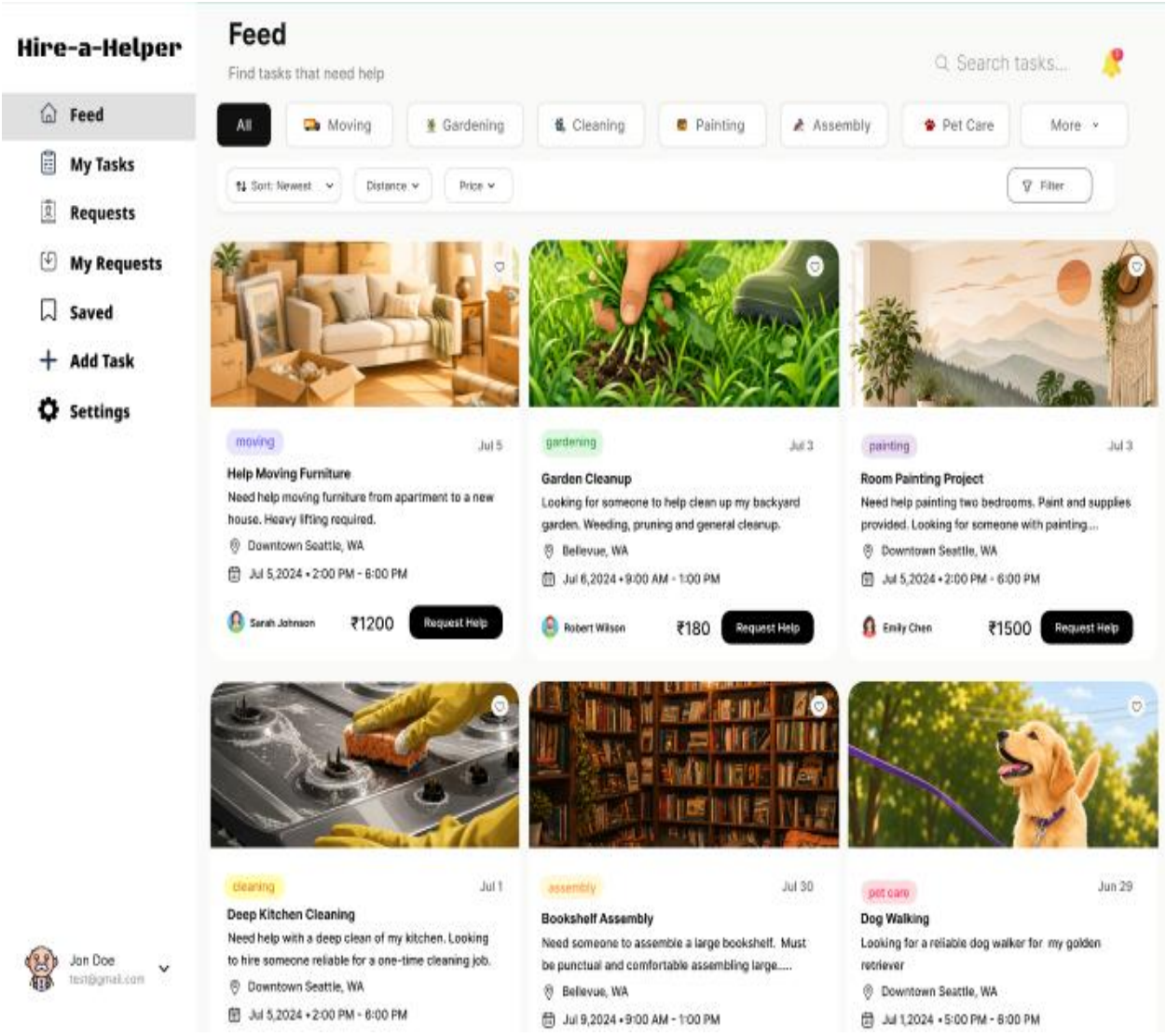
Add Task Interface

Task List Component (Feed)

Displays a list of available tasks for helpers to browse and apply to.

Key Features:

- **Filters:** Allows filtering tasks by category, location, or price.
- **Pagination & Search:** Ensures seamless navigation through large task lists using keywords.



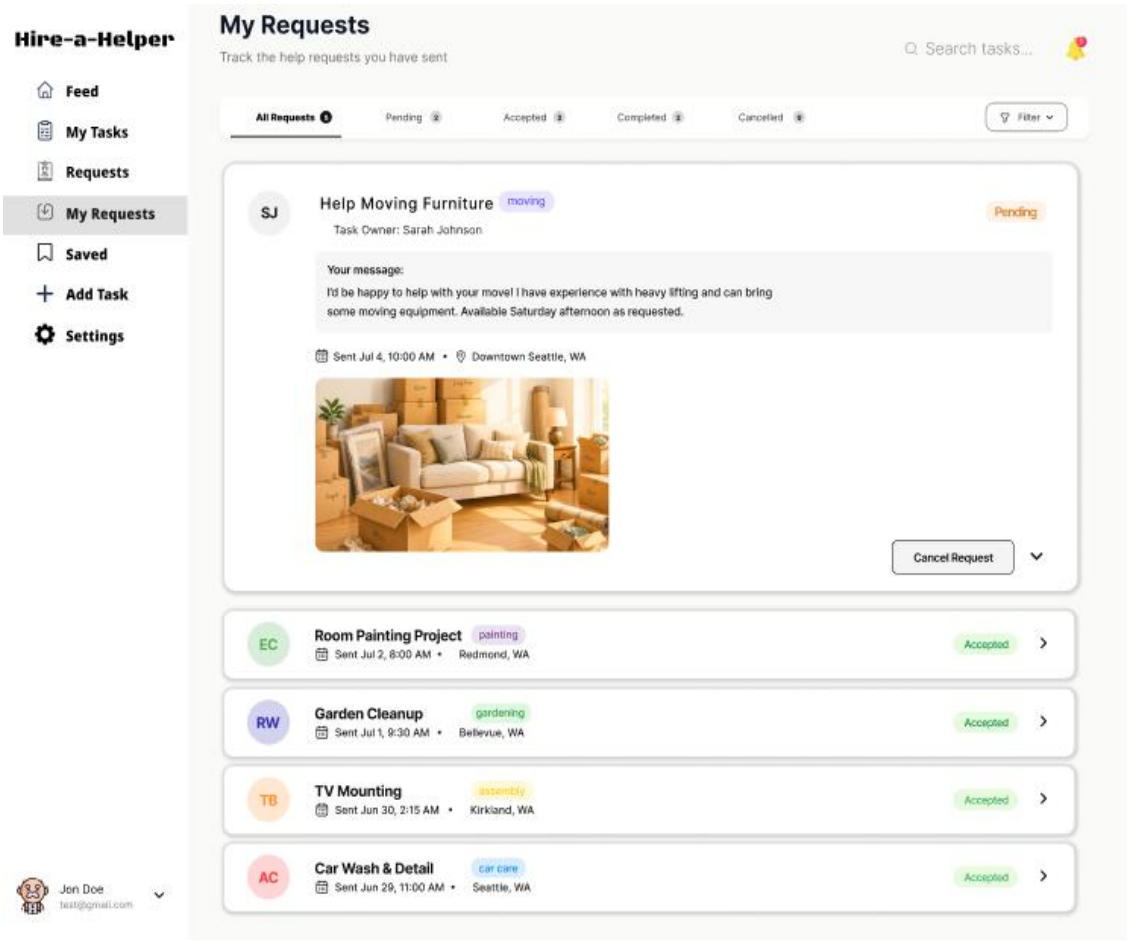
Feed Interface

Helper Application Component (My Requests)

Allows helpers to manage their sent applications.

Key Features:

- **Track Application Status:** Helpers can monitor the progress of their applications (e.g., Pending, Accepted, Declined).
- **View History:** Helpers can see a list of all the tasks they have requested to work on.



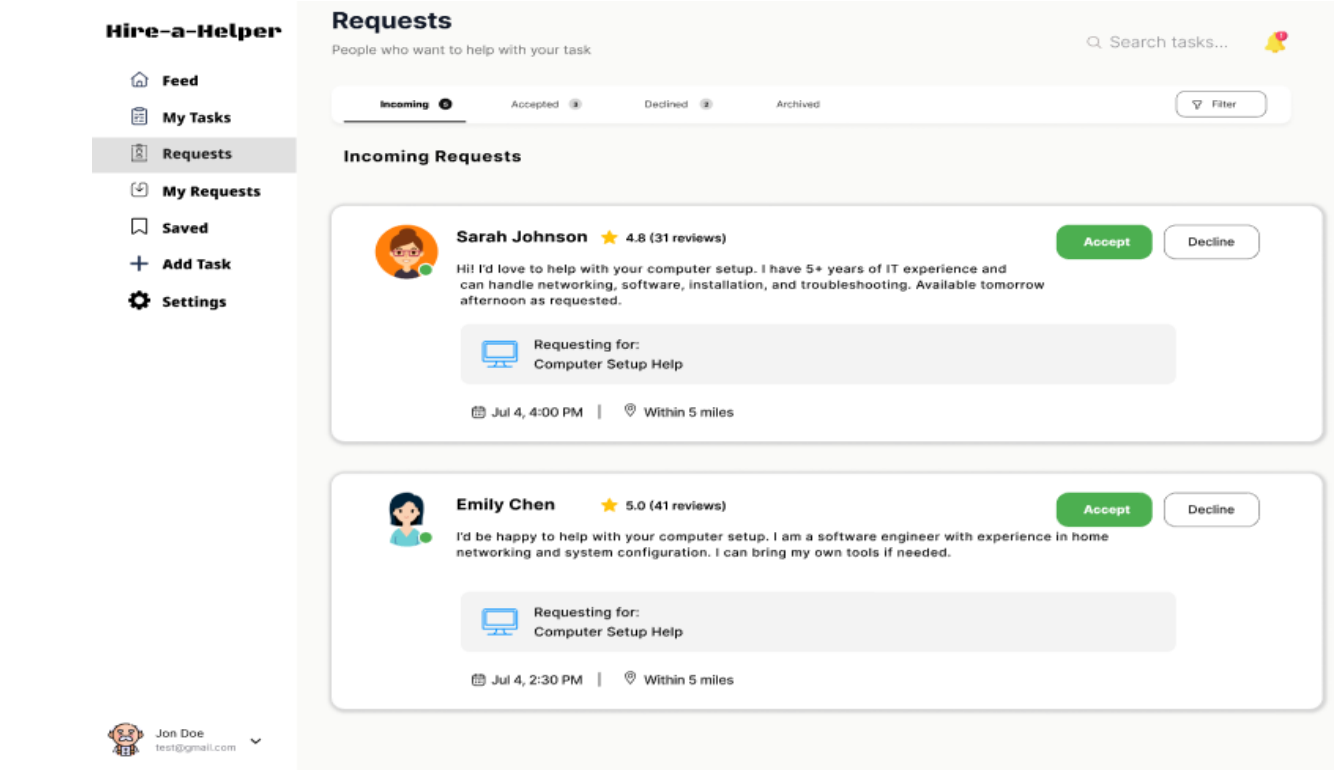
My Request Interface

Application Management Component (Incoming Requests)

Enables task owners to view and manage incoming task applications from helpers.

Key Features:

- **Status Tracking:** Displays the status of applications (e.g., pending, shortlisted).
- **Accept/Decline Application:** Allows the task owner to accept or decline applications if needed.
- **Details View:** Provides detailed helper profiles and application history.



Incoming Request Interface

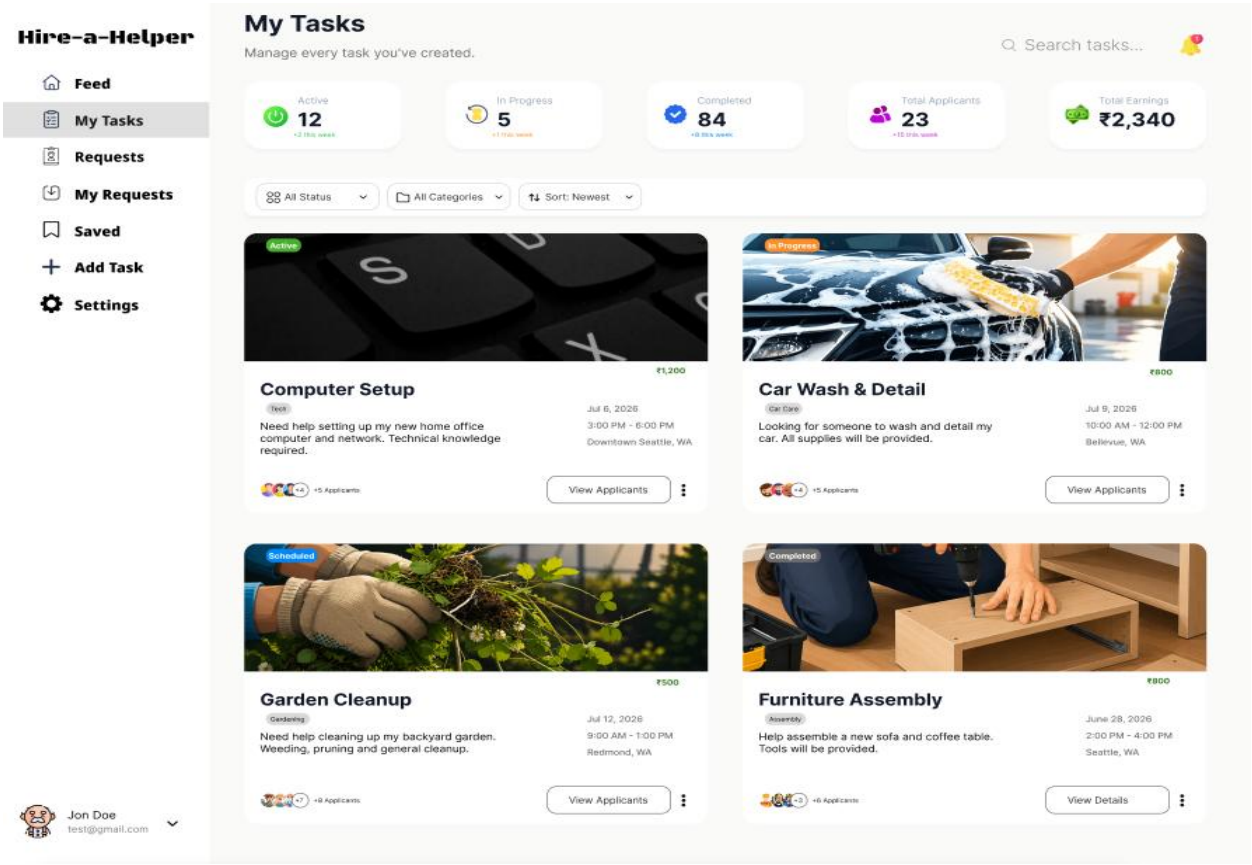
My Tasks Component (Task Management)

Description: A dedicated interface where Task Owners can view, track, and manage all the tasks they have created and posted on the platform.

Key Features:

- **Quick Statistics Overview:** Displays a top summary bar with real-time stats including Active Tasks, Tasks In Progress, Completed Tasks, Total Applicants, and Total Earnings.
- **Advanced Filtering & Sorting:** Allows users to easily filter their tasks using dropdowns for 'Status' (e.g., Active, Completed) and 'Categories', along with sorting options like 'Newest'.
- **Task Status Badges:** Each task card clearly displays a color-coded status badge (e.g., Green for Active, Orange for In Progress, Blue for Scheduled) for quick visual tracking.

- **Applicant Tracking**: Displays small avatar icons showing exactly how many helpers have applied for a specific task.
- **Action Controls:** Includes buttons instantly "View Applicants" or "View Details", and a three-dot menu for editing or deleting the task.



My Task Interface

Notifications Component

Description: An interactive dropdown menu accessible via the top navigation bar, keeping users instantly updated on their task activities and incoming requests.

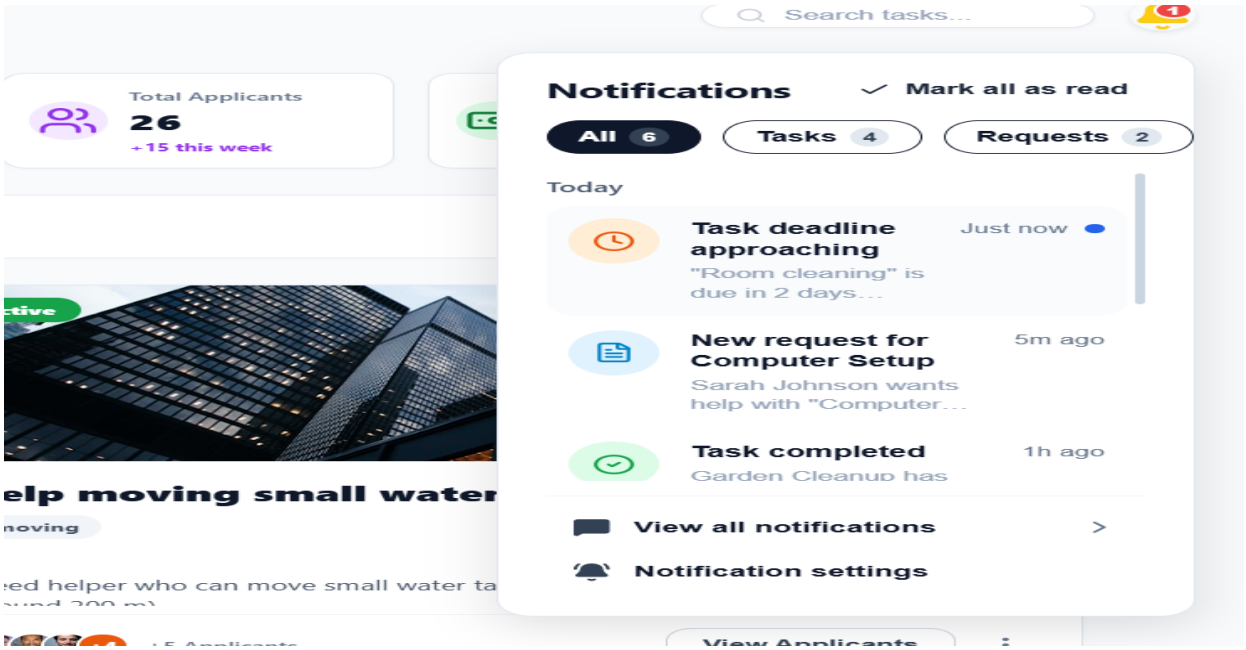
Key Features:

Smart Categorization: Notifications are neatly organized into clickable tabs (All, Tasks, Requests) with live badges showing the count of unread alerts**.**

Real-time Alerts: Displays distinct, icon-based alerts for critical events like "Task deadlines", "New helper requests", and "Task completions".

Quick Actions: Allows users to easily clear their inbox using a one-click "Mark all as read" functionality.

Detailed Timestamps: Shows relative time (e.g., "Just now", "5m ago") along with a blue dot indicator for unread messages, ensuring users never miss an important update**.**



Notifications Interface

API Components

User Authentication & Management

User Registration

- **Route:** POST http://localhost:5000/api/auth/register
- **Description:** Register a new user account.
- **Input:**{
  "firstName": "Aditya",
  "lastName": "Sharma",
  "email": "adityasharma@gmail.com",
  "password": "Password123",
  "phone": "9876543210"
  }
- **Output:**

- `201 Created: {"Message": "Registration Successful." OTP sent to your email."}`
- `Error Output (400 Bad Request): { "success": false, "message": "All fields are required"}`
- Error Output (409 Conflict - User Exists): { "success": false, "message": "User with this email or phone number already exists"}

User Login

- **Route:** `POST` http://localhost:5000/api/auth/login
- **Description:** Authenticate a user.
- **Input:**{

```
"email": "adityasharma@gmail.com",
"password": "Password123"
}
```

- **Output:**
  - `200 OK: {"message": "Login successful", "data": {"token": "jwt_token..."}}`
  - `Error Output (401 Unauthorized - Wrong Credentials):{"success": false,"message": "Invalid email or password"}`
  - Error Output (403 Forbidden - Email Not Verified):{"success": false, "message": "Please verify your email first"}

Password Reset Request

- **Route:** `POST` http://localhost:5000/api/auth/forgot-password
- **Description:** Send OTP for password reset.
- **Input: {**`"email": "adityasharma@gmail.com" }`

- **Output:**
  - `200 OK: {"message": "OTP sent successfully"}`

Verify OTP for Password Reset

- **Route:** `POST` http://localhost:5000/api/auth/verify-otp
- **Description:** Verify OTP sent to email.
- **Input: {**`"email": "adityasharma@gmail.com", "otp": "698214" }`

- **Output:**
  - `200 OK: {"message": "OTP verified successfully"}`
  - `Error Output (400 Bad Request - Expired or Invalid): {"success": false, "message": "Invalid OTP"}`
  - `Error Output (404 Not Found):{ "success": false,"message": "User not found"}`

Task Postings

Create Task Posting

- **Route:** `POST` http://localhost:5000/api/tasks
- **Description:** Create a new task posting.
- **Input (FormData):** `title`, `description`, `location`, `budget`, `image`
- **Output:**
  - `201 Created: { "message": "Task posted successfully" }`

Get All Task Postings (Feed)

- **Route:** `GET` http://localhost:5000/api/tasks/feed
- **Description:** Retrieve all task postings for helpers.
- **Input:** None
- **Output:**
  - `200 OK: [ { "taskId": "1", "title": "Move Furniture", "status": "Active" }, ... ]`

Update Task Posting

- **Route:** `PATCH` http://localhost:5000/api/tasks/:id
- **Description:** Update a task posting.
- **Input:** `id: 673ff1845ca9da30126554c8`
- **Output:**
  - `200 OK: { "message": "Task updated successfully" }`

Delete Task Posting

- **Route:** `DELETE` http://localhost:5000/api/tasks/:id
- **Description:** Delete a specific task posting.
- **Input:** `id: 673ff1845ca9da30126554c8`

- **Output:**
    - `200 OK: { "message": "Task deleted successfully" }`

Task Applications / Requests

Apply for a Task (Send Request)

- **Route:** `POST` http://localhost:5000/api/requests
- **Description:** This route allows a helper to submit a task request.
- **Input:**`{ "taskId": "673ff1845ca9da30126554c8" }`

- **Output:**
    - `201 Created: { "message": "Request submitted successfully" }`

Get Application by ID

- **Route:** `GET` http://localhost:5000/api/requests/incoming
- **Description:** Retrieve all incoming requests for the user's tasks.
- **Input:** None
- **Output:**
    - `200 OK: [ { "requestId": "1", "helperId": "2", "status": "Pending" } ]`

Update Application Status

- **Route:** `PATCH` http://localhost:5000/api/requests/:id/accept
- **Description:** Update the status of an application to accepted.
- **Input:** `id` of the request
- **Output:**
    - `200 OK: { "message": "Application status updated" }`

Error Handling and Troubleshooting

Effective error handling and troubleshooting ensure the smooth operation of the Hire-a-Helper platform. Below are some common errors that users and developers may encounter.

Gmail SMTP Not Working

Error Description: The platform uses SMTP to send email notifications (e.g., OTPs, confirmation emails). If SMTP fails, emails won't be sent. **Troubleshooting Steps:**

- **Check Gmail Settings:** Ensure that you are using an **App Password**. Google blocks connections from regular passwords for security.
- **Check Credentials:** Double-check the email and App Password provided in the `.env` configuration to ensure they are correct and have no extra spaces.
- **Check SMTP Settings:** Server `smtp.gmail.com`, Port `587` (for TLS). Ensure the secure flag is set correctly.

Port Conflicts

Error Description: The application may fail to start due to port conflicts (e.g., the server tries to bind to port 5000 or 5173, but it's already being used). **Troubleshooting Steps:**

- **Check Which Service is Using the Port:** On Windows, use:`netstat -ano | findstr :5000`
- **Stop the Conflicting Process:** Find the PID and run:`taskkill /PID <PID> /F`
- **Change the Port:** Update your `.env` file to use a new port number (e.g., 5001 or 8080).

Database Connection Failures

Error Description: The application may fail to connect to the database (`ECONNREFUSED`). **Troubleshooting Steps:**

- **Check Database Credentials:** Ensure that the database username, password, and connection string in the configuration are correct.
- **Check Database Access Permissions:** For MongoDB Atlas, ensure your IP address is whitelisted

File Upload Failures

- **Error Description:** Users may encounter issues when uploading profile images or task photos. **Troubleshooting Steps:**
- **Check File Size Limits:** Ensure the file size does not exceed the server's upload limits configured in Multer.
- **Verify File Type:** Ensure that only allowed file types (e.g., .jpg, .png) are uploaded.

Maintenance and Support

• Procedures for Regular Maintenance

- **Backing Up the Database:** Use MongoDB Atlas's built-in backup tools to schedule regular backups to prevent data loss.
- **Rotating API Keys or Credentials:** Regularly update sensitive credentials stored in environment variables (e.g., `.env` files).
- **Monitoring Logs and Server Health:** Always check the terminal logs for detailed error messages. They can provide more context on what's going wrong.
- in the Network Access settings.

API Request Failures

Error Description: API requests may fail due to misconfiguration or incorrect endpoint usage.
Troubleshooting Steps:

- **Check API Authentication:** Ensure that JWT tokens are correctly included in the request headers (`Authorization: Bearer <token>`).
- **Examine Response Codes:** 404 (Not Found), 500 (Internal Server Error), 401 (Unauthorized).

Version Control and Updates

Using Git for Version Control

Branching Strategies:

- Use the `main` branch for production-ready code.
- Create feature branches for new features or bug fixes.
- Use release branches for testing updates before merging into the main branch.

Commit Messages: Follow a standard format such as `[feature]: Add user authentication` or `[fix]: Resolve sign in  bug`.

Applying Updates and Migrations Safely: Test updates in a staging environment before deploying to production.

Glossary or FAQ

Glossary

- **Matching Algorithm:** A system that matches helpers to tasks based on their categories, budget, and location.
- **Helper Profile:** A section in a user profile where professionals showcase their personal details.
- **Availability Tracking:** A feature that allows task owners to view the real-time application status of helpers.

FAQs

- **Why is the Gmail SMTP not working?** Ensure that you have generated a Google App Password and 2-Step Verification is enabled.
- **What to do if I encounter a port conflict?** Use the command `netstat -ano | findstr :5000` to identify the service occupying the port and stop it.
- **How can I reset my password?** Go to the login page and click on "Forgot Password." Follow the instructions to reset it via email OTP.

Account and Profiles

- **How can I update my profile details?** Sign in to your account, navigate to the Settings section in the dashboard, and edit your details to make updates.

Tasks and Applications

- **How do I post a Task as a user?** Navigate to the Add Task section, fill in the Task details (e.g., description, budget, and location), and submit it.
- **Can I apply for multiple Tasks at the same time?** Yes, helpers can apply for multiple tasks by sending requests to different task owners.

Technical Issues

- **Why am I unable to upload my profile picture?** Ensure that your files meet the size and format requirements (e.g., PNG, JPG, under 5MB).
- **I'm not receiving email notifications. What should I do?** If `EMAIL_USER` is empty in the `.env` file, the backend runs in development mode and prints OTPs to the console instead of sending them. Check your terminal logs.

Future Enhancements

- **Integration with Third-Party Tools:** Add integration with tools like WhatsApp or SMS for instant mobile notifications.
- **AI-Driven Recommendations:** Implement AI algorithms to recommend the best helpers or tasks based on past trends and ratings.
- **Enhanced Reporting and Analytics:** Provide detailed insights into hiring trends, task completion rates, and user engagement through an Admin dashboard.
- **Payment Gateway Integration:** Allow users to pay helpers securely directly through the platform using Stripe or Razorpay.